

## Week 2 — Practice Problems — Solutions

Day 2 Live-Coding Session — 5 Problems

**Name:** Gursimran Bajwa | **Registration No:** RA2511003011289

**Language:** Java | **Problems solved:** 5 / 5

### Problem 1 — Vowel & Consonant Counter

#### Problem Statement

A library orientation kiosk counts the vowels and consonants in a submitted book title for a simple text-stats display.

#### Approach

- Each character is read with `charAt(i)` and lowercased once with `Character.toLowerCase()`, which makes the whole comparison case-insensitive without duplicating conditions.
- Spaces are skipped with a `continue` statement.
- A character is a vowel if it matches `a`, `e`, `i`, `o` or `u`; otherwise it is counted as a consonant only when it lies in the `'a'–'z'` range, so digits and punctuation are safely ignored.
- The two counters are printed in the required single-line format.

#### Java Program — `VowelConsonantCounter.java`

```
/**
 * Problem 1 - Vowel & Consonant Counter (Library Orientation Kiosk)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class VowelConsonantCounter {

    /* Walks the text with charAt() and buckets each letter as a vowel or consonant. */
    public static void countVowelsAndConsonants(String text) {
        int vowels = 0, consonants = 0;

        for (int i = 0; i < text.length(); i++) {
            char ch = Character.toLowerCase(text.charAt(i));

            if (ch == ' ') continue; // spaces are ignored

            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                vowels++;
            } else if (ch >= 'a' && ch <= 'z') {
                consonants++;
            }
        }

        System.out.println("Vowels: " + vowels + " | Consonants: " + consonants);
    }

    public static void main(String[] args) {
        String[] titles = { "Java Programming", "Data Structures", "Gursimran Bajwa" };

        for (String title : titles) {
            System.out.println("Title: \"" + title + "\"");
            countVowelsAndConsonants(title);
            System.out.println();
        }
    }
}
```

#### Output

```
Title: "Java Programming"
Vowels: 5 | Consonants: 10
```

Title: "Data Structures"  
Vowels: 5 | Consonants: 9

Title: "Gursimran Bajwa"  
Vowels: 5 | Consonants: 9

## Problem 2 — CSV Student Record Parser

### Problem Statement

The T&P team receives student registration data as CSV lines of the form "Name,RollNumber,Department" and needs a parser that splits each line into fields and prints a formatted record.

### Approach

- `split(",")` breaks the line into a `String` array.
- The array length is validated first — anything other than exactly 3 fields prints "Invalid Record" and returns early, so no index-out-of-bounds can occur.
- Each field is `trim()`-ed to tolerate spaces around the commas.
- The record is printed by concatenating the three fields into the required "Name: ... | Roll No: ... | Dept: ..." layout.

### Java Program — `CsvStudentRecordParser.java`

```
/**
 * Problem 2 - CSV Student Record Parser (T&P Registration Data)
 * Name: Gursimran Bajwa   |   Reg. No: RA2511003011289
 */
public class CsvStudentRecordParser {

    /* Splits a CSV line into exactly 3 fields and prints a formatted record. */
    public static void parseStudentRecord(String csvLine) {
        String[] fields = csvLine.split(",");

        if (fields.length != 3) {
            System.out.println("Invalid Record");
            return;
        }

        String name      = fields[0].trim();
        String rollNumber = fields[1].trim();
        String department = fields[2].trim();

        System.out.println("Name: " + name + " | Roll No: " + rollNumber + " | Dept: " + department);
    }

    public static void main(String[] args) {
        String[] lines = {
            "Ananya Verma,RA2211003010123,CSE",
            "Ananya Verma,CSE",
            "Gursimran Bajwa,RA2511003011289,CSE"
        };

        for (String line : lines) {
            System.out.println("Input: \"" + line + "\"");
            parseStudentRecord(line);
            System.out.println();
        }
    }
}
```

### Output

```
Input: "Ananya Verma,RA2211003010123,CSE"
Name: Ananya Verma | Roll No: RA2211003010123 | Dept: CSE

Input: "Ananya Verma,CSE"
Invalid Record

Input: "Gursimran Bajwa,RA2511003011289,CSE"
Name: Gursimran Bajwa | Roll No: RA2511003011289 | Dept: CSE
```

## Problem 3 — File Extension Validator

### Problem Statement

An assignment-upload portal must check whether an uploaded filename carries an accepted extension (pdf, docx, zip), regardless of case, before accepting the submission.

### Approach

- `lastIndexOf('.')` finds the final dot, which is what correctly handles names such as "my.report.pdf".
- Two edge cases are rejected up front: no dot at all, and a dot as the final character (nothing after it to read).
- `substring(dotIndex + 1)` extracts the extension, which is compared against the accepted list with `equalsIgnoreCase()` so "PDF" and "pdf" both pass.
- Test cases cover an uppercase extension, a rejected type and a file with no extension at all.

### Java Program — FileExtensionValidator.java

```
/**
 * Problem 3 - File Extension Validator (Assignment Upload Portal)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class FileExtensionValidator {

    static final String[] ACCEPTED = { "pdf", "docx", "zip" };

    /* Extracts the extension after the last dot and matches it case-insensitively. */
    public static String validateFileExtension(String filename) {
        int dotIndex = filename.lastIndexOf('.');

        if (dotIndex == -1 || dotIndex == filename.length() - 1) {
            return "Rejected - invalid file type"; // no dot, or nothing after it
        }

        String extension = filename.substring(dotIndex + 1);

        for (String accepted : ACCEPTED) {
            if (extension.equalsIgnoreCase(accepted)) {
                return "Accepted";
            }
        }
        return "Rejected - invalid file type";
    }

    public static void main(String[] args) {
        String[] uploads = { "Assignment1.PDF", "notes.txt", "report.docx",
                             "archive.ZIP", "README" };

        for (String filename : uploads) {
            System.out.println(filename + " -> " + validateFileExtension(filename));
        }
    }
}
```

### Output

```
Assignment1.PDF -> Accepted
notes.txt -> Rejected - invalid file type
report.docx -> Accepted
archive.ZIP -> Accepted
README -> Rejected - invalid file type
```

## Problem 4 — Masked Phone Number Formatter

### Problem Statement

A student-support call centre displays a partially masked phone number on screen for privacy, while agents confirm identity using the last 4 digits.

### Approach

- Validation runs in two stages: the length must be exactly 10, and every character must pass `Character.isDigit()`.
- A `StringBuilder` is seeded with the six mask characters, then `substring(6)` appends the visible last four digits.
- `insert(6, "-")` places the separator between the mask and the visible digits, which is the `StringBuilder` operation the problem asks for.
- Invalid inputs (too short, and one containing letters) return the error message instead.

### Java Program — `MaskedPhoneNumberFormatter.java`

```
/**
 * Problem 4 - Masked Phone Number Formatter (Student Support Call Center)
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 */
public class MaskedPhoneNumberFormatter {

    /* Validates a 10-digit number and builds "XXXXXX-<last 4 digits>". */
    public static String maskPhoneNumber(String phone) {
        if (phone == null || phone.length() != 10) {
            return "Invalid phone number";
        }

        for (int i = 0; i < phone.length(); i++) {
            if (!Character.isDigit(phone.charAt(i))) {
                return "Invalid phone number";
            }
        }

        StringBuilder masked = new StringBuilder("XXXXXX");
        masked.append(phone.substring(6)); // the visible last 4 digits
        masked.insert(6, "-"); // separator for readability

        return masked.toString();
    }

    public static void main(String[] args) {
        String[] numbers = { "9876543210", "98765", "98765abc10", "9123456789" };

        for (String phone : numbers) {
            System.out.println(phone + " -> " + maskPhoneNumber(phone));
        }
    }
}
```

### Output

```
9876543210 -> XXXXXX-3210
98765 -> Invalid phone number
98765abc10 -> Invalid phone number
9123456789 -> XXXXXX-6789
```

## Problem 5 — Bank Transaction Reference Generator & Validator

### Problem Statement

A fintech onboarding module must normalise and validate 14-character transaction references: 3 letters (bank code) + 6 digits (ddMMyy) + 5 digits (sequence). Pasted codes may carry stray spaces or a mixed-case bank code.

### Approach

- `normalizeReference()` trims the input, then rebuilds it as `substring(0,3).toUpperCase() + substring(3)` so only the bank code changes case and the digits are left untouched.
- Validation runs in three ordered stages so the error message can name the exact reason: length must be 14, characters 0–2 must pass `Character.isLetter()`, and characters 3–13 must pass `Character.isDigit()`. No regex is used.
- On success, `substring()` slices out the bank code, day, month, year and sequence, and a `StringBuilder` assembles the "[BANK] DATE: dd/MM/yy | SEQ: nnnnn" display line.
- Five test references exercise the valid path and all three distinct failure reasons.

### Java Program — BankTransactionReference.java

```
/**
 * Problem 5 - Bank Transaction Reference Generator & Validator
 * Name: Gursimran Bajwa | Reg. No: RA2511003011289
 * Valid reference = 3 letters (bank code) + 6 digits (ddMMyy) + 5 digits (sequence) = 14 chars.
 */
public class BankTransactionReference {

    /* Trims stray spaces and uppercases only the 3-character bank code. */
    public static String normalizeReference(String raw) {
        String trimmed = raw.trim();

        if (trimmed.length() < 3) {
            return trimmed; // too short to have a bank code
        }
        return trimmed.substring(0, 3).toUpperCase() + trimmed.substring(3);
    }

    /* Runs the three validation stages and formats the reference when it passes. */
    public static String validateAndFormat(String reference) {
        if (reference.length() != 14) {
            return "Invalid: reference must be exactly 14 characters";
        }

        for (int i = 0; i < 3; i++) {
            if (!Character.isLetter(reference.charAt(i))) {
                return "Invalid: bank code must be 3 letters";
            }
        }

        for (int i = 3; i < 14; i++) {
            if (!Character.isDigit(reference.charAt(i))) {
                return "Invalid: date and sequence must be digits";
            }
        }

        String bankCode = reference.substring(0, 3);
        String day = reference.substring(3, 5);
        String month = reference.substring(5, 7);
        String year = reference.substring(7, 9);
        String sequence = reference.substring(9);

        StringBuilder display = new StringBuilder();
        display.append("[").append(bankCode).append("] DATE: ")
            .append(day).append("/").append(month).append("/").append(year)
            .append(" | SEQ: ").append(sequence);

        return display.toString();
    }

    public static void main(String[] args) {
        String[] rawReferences = {
```

```
        " hdf03022600042 ",
        "12F03022600042",
        "sbi1503260012",
        "icI2908260099x",
        " axs29082600777 "
    };

    for (String raw : rawReferences) {
        String normalized = normalizeReference(raw);
        System.out.println("Raw: \"" + raw + "\"");
        System.out.println(" Normalized: \"" + normalized + "\"");
        System.out.println(" Result: " + validateAndFormat(normalized));
        System.out.println();
    }
}
```

## Output

```
Raw: " hdf03022600042 "
Normalized: "HDF03022600042"
Result: [HDF] DATE: 03/02/26 | SEQ: 00042

Raw: "12F03022600042"
Normalized: "12F03022600042"
Result: Invalid: bank code must be 3 letters

Raw: "sbi1503260012"
Normalized: "SBI1503260012"
Result: Invalid: reference must be exactly 14 characters

Raw: "icI2908260099x"
Normalized: "ICI2908260099x"
Result: Invalid: date and sequence must be digits

Raw: " axs29082600777 "
Normalized: "AXS29082600777"
Result: [AXS] DATE: 29/08/26 | SEQ: 00777
```